Question 1a

In [189…

```
import numpy as np
import pandas as pd
from scipy.io import mmread

# Path to the data
data_dir = "filtered_gene_bc_matrices/GRCh38"

# Load genes and cell barcodes
genes = pd.read_csv(
    f"{data_dir}/genes.tsv",
    sep="\t",
    header=None
)

barcodes = pd.read_csv(
    f"{data_dir}/barcodes.tsv",
    sep="\t",
    header=None
)

# Load sparse matrix
matrix = mmread(f"{data_dir}/matrix.mtx")

print("Sparse matrix shape:", matrix.shape)
print("Number of genes:", len(genes))
print("Number of cells:", len(barcodes))
```

```
Sparse matrix shape: (33694, 4340)
Number of genes: 33694
Number of cells: 4340
```

In [190…

```
# Construct a dense matrix of genes by cells from the sparse list
X = matrix.toarray()
print("Dense matrix shape:", X.shape)
print("Number of gene rows:", X.shape[0])
print("Number of cell barcodes:", X.shape[1])
print("Genes detected in at least one cell:", np.count_nonzero(X.sum(axis=1)))
print("Cells with nonzero total counts:", np.count_nonzero(X.sum(axis=0)))
```

```
Dense matrix shape: (33694, 4340)
Number of gene rows: 33694
Number of cell barcodes: 4340
Genes detected in at least one cell: 19773
Cells with nonzero total counts: 4340
```

The dense matrix has shape (33,694, 4,340), with genes as rows and cells as columns. It contains 33,694 gene rows and 4,340 cell barcodes. Of these genes, 19,773 have nonzero expression in at least one cell; the remaining 13,921 genes have zero counts across all cells. All 4,340 cells have nonzero total counts.

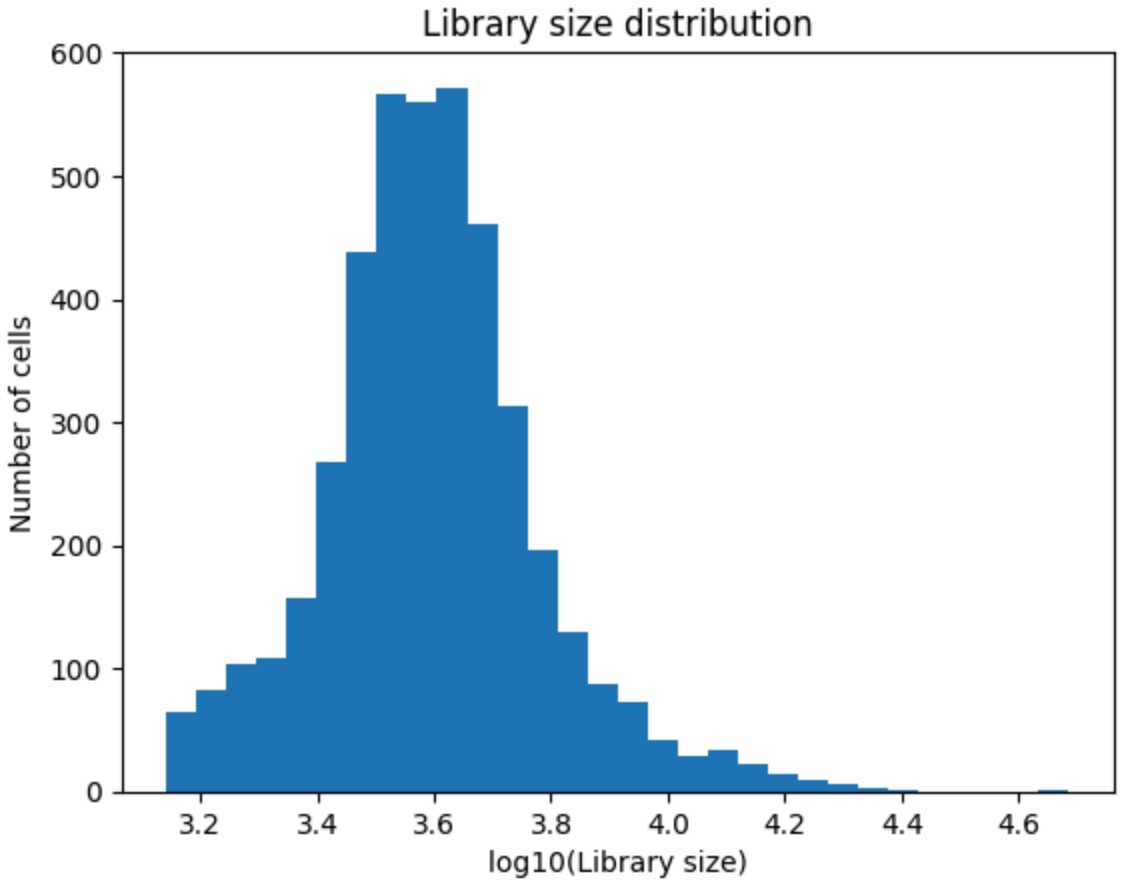
Question 1b

In [191…
```python
import matplotlib.pyplot as plt
# Library size = the total number of expression counts across all genes in one
# Expression = how much expression of a particular gene was detected in a cell

# Total counts for each cell
library_size = X.sum(axis=0)

# Log10 library size
log_library_size = np.log10(library_size)

# Plot histogram
plt.hist(log_library_size, bins=30)
plt.xlabel("log10(Library size)")
plt.ylabel("Number of cells")
plt.title("Library size distribution")
plt.show()
```



In [192…
```python
Q1 = np.percentile(log_library_size, 25)
Q3 = np.percentile(log_library_size, 75)

IQR = Q3 - Q1
threshold = Q1 - 1.5 * IQR

print("Filtering threshold:", threshold)
print("Cells removed:", (log_library_size < threshold).sum())
```

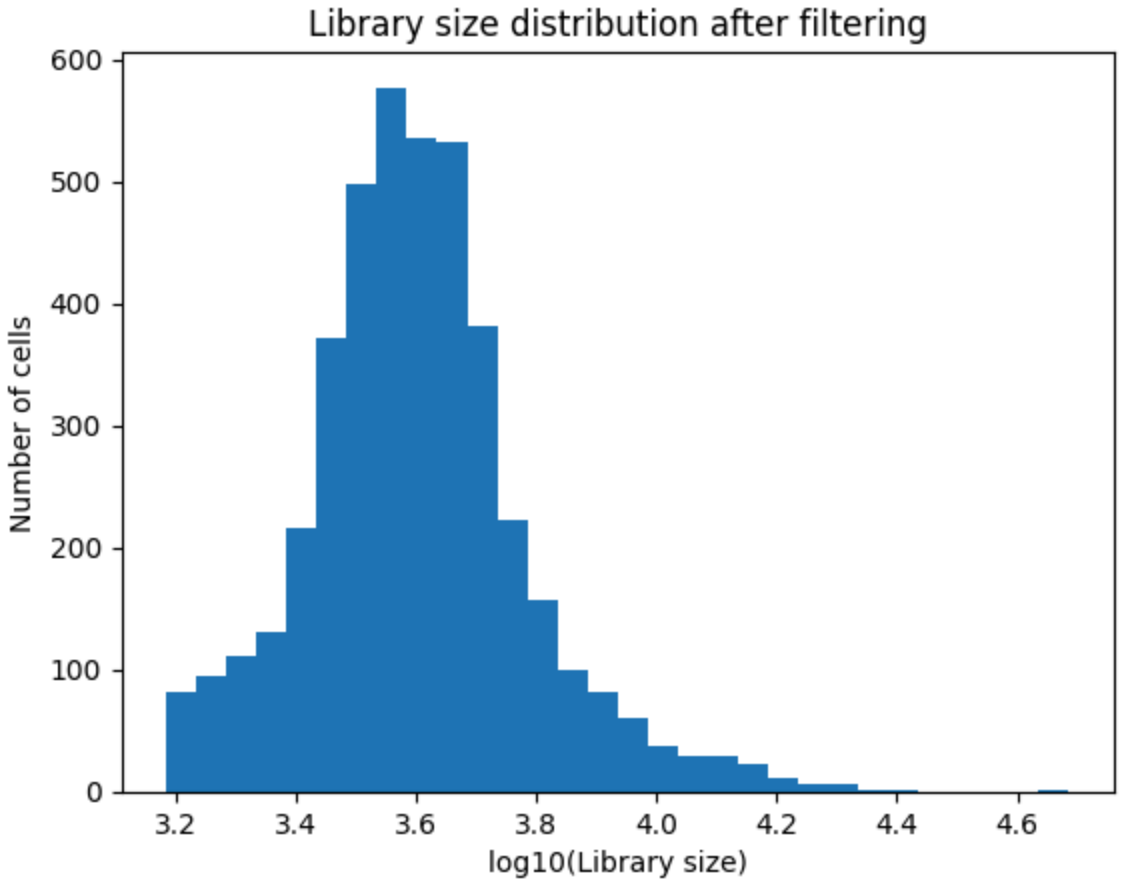
```
Filtering threshold: 3.181986631906989
Cells removed: 48
```

In [193…
```python
# Filter cells
keep = log_library_size >= threshold
```

```
X_filtered = X[:, keep]

# Library sizes after filtering
library_size_filtered = X_filtered.sum(axis=0)
log_library_size_filtered = np.log10(library_size_filtered)

# Plot after filtering
plt.hist(log_library_size_filtered, bins=30)
plt.xlabel("log10(Library size)")
plt.ylabel("Number of cells")
plt.title("Library size distribution after filtering")
plt.show()
```



In [194…

```
median_library_size = np.median(library_size_filtered)

print("Cells remaining:", X_filtered.shape[1])
print("Median library size:", median_library_size)
```

```
Cells remaining: 4292
Median library size: 3892.0
```

In [195…

```
# sensitivity analysis
threshold_counts = 10**threshold

threshold_more = threshold_counts * 1.20
threshold_less = threshold_counts * 0.80

keep_more = library_size >= threshold_more
keep_less = library_size >= threshold_less
```

```
median_more = np.median(library_size[keep_more])
median_less = np.median(library_size[keep_less])

print("Original threshold:", threshold_counts)
print("Original median:", median_library_size)

print("20% more stringent:")
print("Cells removed:", (~keep_more).sum())
print("Median:", median_more)

print("20% less stringent:")
print("Cells removed:", (~keep_less).sum())
print("Median:", median_less)
```

```
Original threshold: 1520.5007262182596
Original median: 3892.0
20% more stringent:
Cells removed: 184
Median: 3949.0
20% less stringent:
Cells removed: 0
Median: 3866.5
```

The log10 library-size distribution is unimodal, with most cells concentrated around the main peak and a small tail of cells with lower library sizes. The log10 library sizes are approximately bell-shaped, suggesting a Gaussian distribution as a rough fit on the log10 scale, or equivalently a log-normal distribution for the original library sizes.

I filtered cells with unusually low library sizes because they contain less expression data and may be low-quality. I used the lower Tukey fence, Q_1-1.5(IQR), rather than using a threshold from Scanpy or Seurat. This gave a cutoff of log10(library size)=3.182, or about 1,521 counts. This removed 48 cells, leaving 4,292 cells. After filtering, the median library size was 3,892 counts.

For the sensitivity analysis, a 20% more stringent threshold removed 184 cells and gave a median of 3,949, while a 20% less stringent threshold removed 0 cells and gave a median of 3,866.5. The median therefore increased by about 1.46% and decreased by about 0.66%, respectively. This suggests that the original filtering decision was not very consequential for the median library size, although a stricter cutoff did remove substantially more cells.

Question 1c

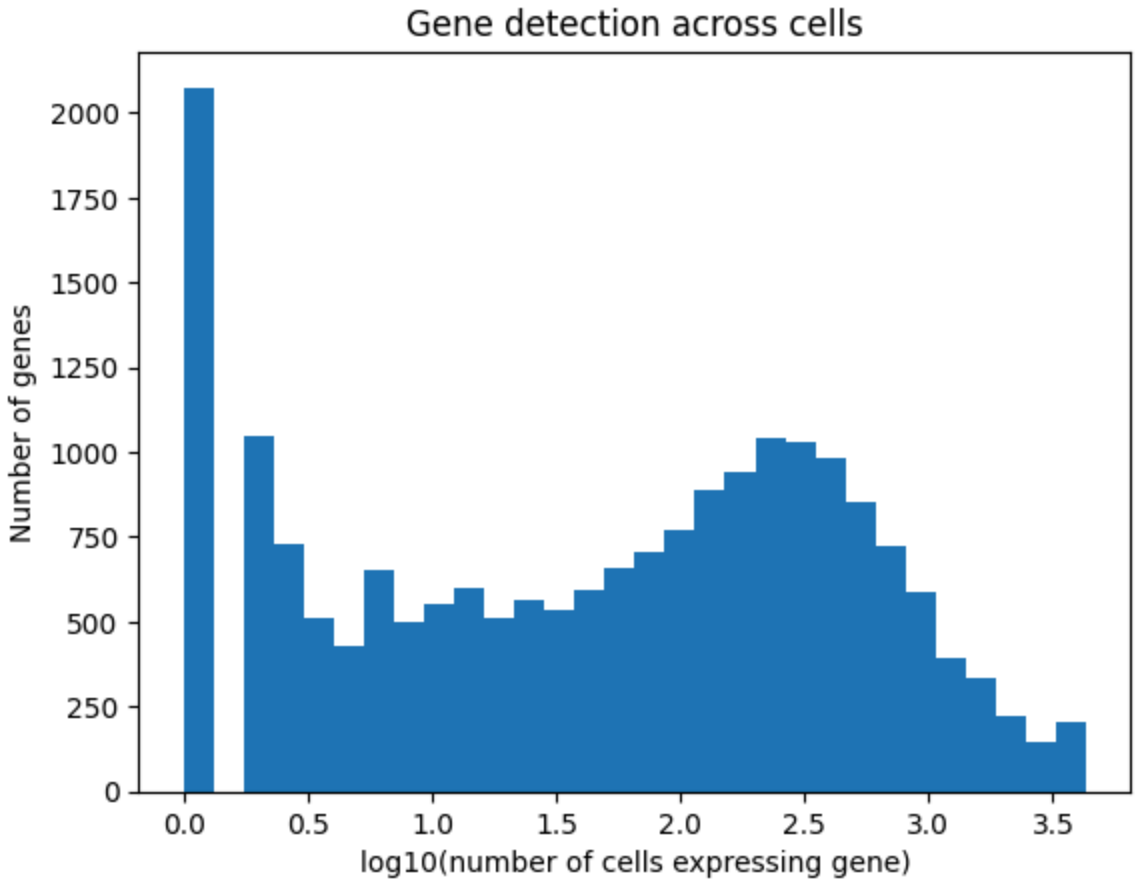
In [196…

```
gene_cells = (X_filtered > 0).sum(axis=1)

gene_cells_detected = gene_cells[gene_cells > 0]

log_gene_cells = np.log10(gene_cells_detected)

plt.hist(log_gene_cells, bins=30)
plt.xlabel("log10(number of cells expressing gene)")
plt.ylabel("Number of genes")
plt.title("Gene detection across cells")
plt.show()
```



In [197…
```
# Check how many genes are detected in the least number of cells
for n in range(1, 11):
    print(n, (gene_cells == n).sum())
```

```
1 2075
2 1046
3 728
4 513
5 430
6 335
7 318
8 279
9 218
10 202
```

In [198…
```
# Retain genes detected in at least two cells.
keep_genes = gene_cells > 1

print("Undetected genes removed:", (gene_cells == 0).sum())
print("Single genes removed as potential noise:", (gene_cells == 1).sum())
print("Total genes removed:", (~keep_genes).sum())
print("Genes remaining:", keep_genes.sum())
```

```
Undetected genes removed: 13933
Single genes removed as potential noise: 2075
Total genes removed: 16008
Genes remaining: 17686
```

The histogram looks multimodal, with a sharp peak at $\log_{10}(1) = 0$ and a broader peak around $\log_{10}(n) = 2.3$–2.6, which corresponds to about 200–400 cells. Looking at the

low-count genes, 2,075 are detected in only one cell, 1,046 in two cells, and 728 in three cells.

I removed genes detected in fewer than two cells. This removes the lowest peak, while keeping genes that are detected in more than one cell and could still represent rare cell populations. This cutoff is based on the observed distribution, although detecting a gene in only one cell does not necessarily mean it is noise.

After cell filtering, 13,933 genes were not detected in any cells, and 2,075 were detected in exactly one cell. Removing these leaves 17,686 genes for further analysis. Genes detected in zero cells were not included in the histogram because $\log_{10}(0)$ is undefined.

Question 2a

In [199…

```
# Apply the gene filtering from Q1c
keep_genes = gene_cells > 1
X_final = X_filtered[keep_genes, :]

# Calculate the library size of each cell
library_size_final = X_final.sum(axis=0)

# Calculate the median library size
median_library_size = np.median(library_size_final)

print("Final matrix shape:", X_final.shape)
print("Median library size:", median_library_size)

# Normalize each cell to the median library size
X_normalized = X_final * (median_library_size / library_size_final)

# Checking
print("First 10 normalized library sizes:")
print(X_normalized.sum(axis=0)[:10])
```

```
Final matrix shape: (17686, 4292)
Median library size: 3892.0
First 10 normalized library sizes:
[3892. 3892. 3892. 3892. 3892. 3892. 3892. 3892. 3892. 3892.]
```

Question 2b

In [200…

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Transpose so cells are rows and genes are columns
X_cells = X_normalized.T

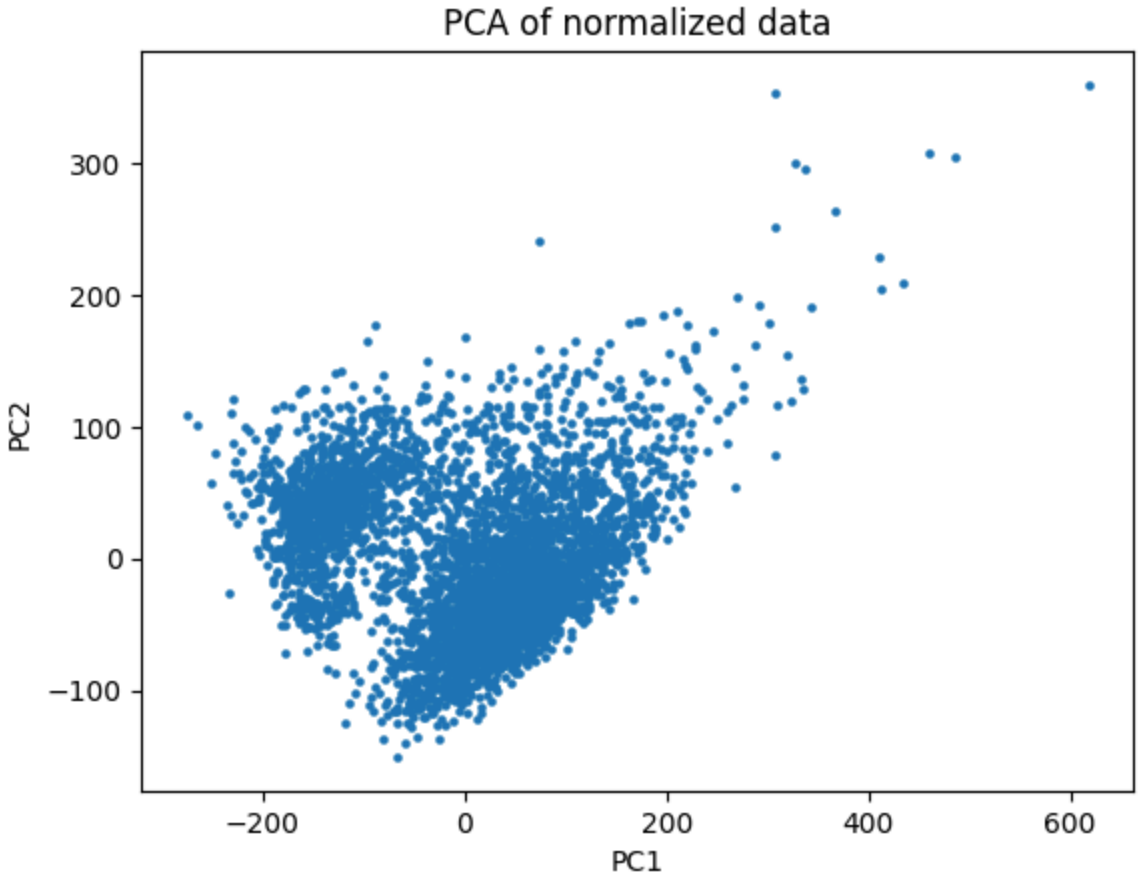
# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_cells)
```

```
print("PCA shape:", X_pca.shape)
print("Variance explained:", pca.explained_variance_ratio_)
```

```
PCA shape: (4292, 2)
Variance explained: [0.44678079 0.15674056]
```

In [201…

```
# Plot the first two components
plt.scatter(X_pca[:, 0], X_pca[:, 1], s=5)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA of normalized data")
plt.show()
```



In [202…

```
# Run t-SNE
from sklearn.manifold import TSNE

tsne = TSNE(
    n_components=2,
    random_state=42,
    perplexity=30
)

X_tsne = tsne.fit_transform(X_cells)

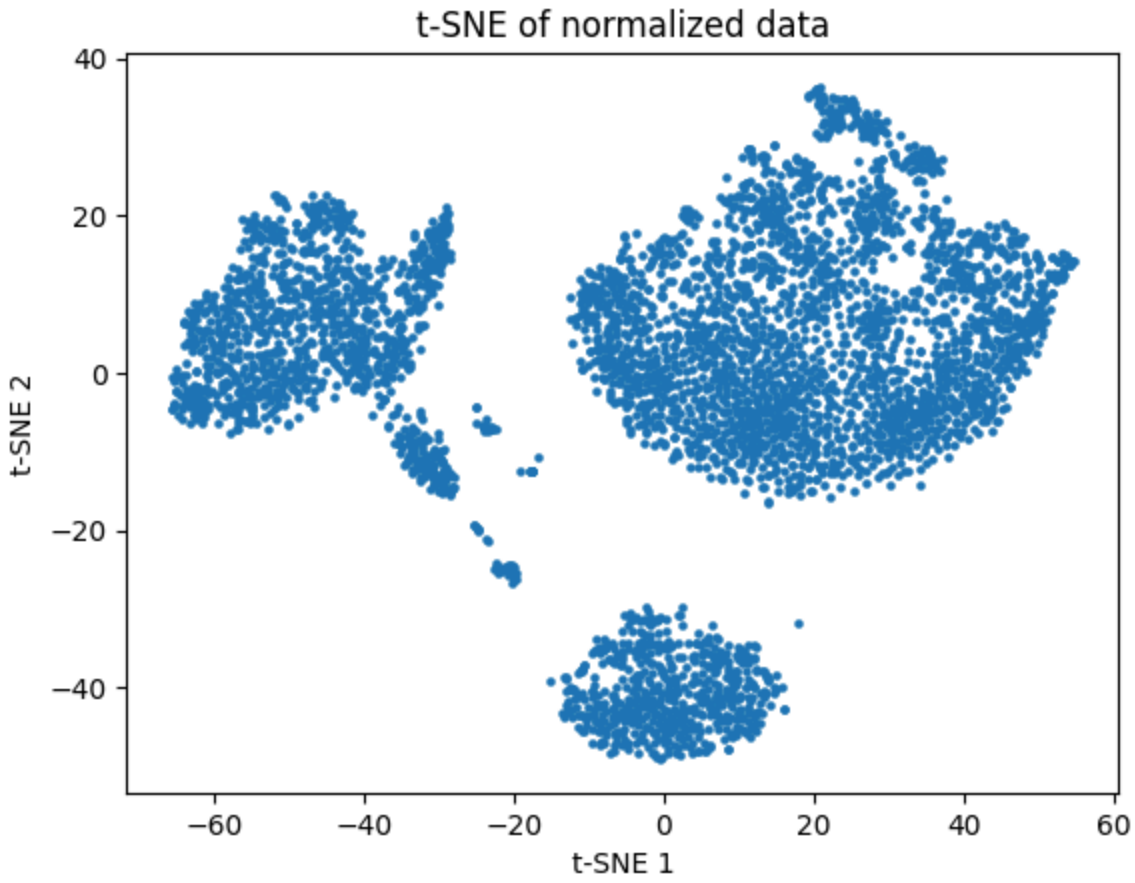
print("t-SNE shape:", X_tsne.shape)
```

```
t-SNE shape: (4292, 2)
```

In [203…

```
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], s=5)
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
```

```
plt.title("t-SNE of normalized data")
plt.show()
```



I used t-SNE to embed the normalized data into two dimensions and compared it with the first two principal components. PC1 explained 44.68% of the variance and PC2 explained 15.67%, for a total of 60.35%. The PCA plot shows a broad distribution of cells with substantial overlap, while the t-SNE plot shows several more clearly separated groups. This difference occurs because PCA is a linear method that captures the largest sources of overall variation, whereas t-SNE is a nonlinear method that focuses more on preserving local relationships between cells. t-SNE therefore can reveal local groups that are less obvious in the PCA plot. One assumption of t-SNE is that nearby cells in the high-dimensional data should remain nearby in the 2D embedding. The t-SNE result also depends on parameters such as perplexity and random initialization, so distances between widely separated groups should not be interpreted as meaningful global distances.

Question 2c

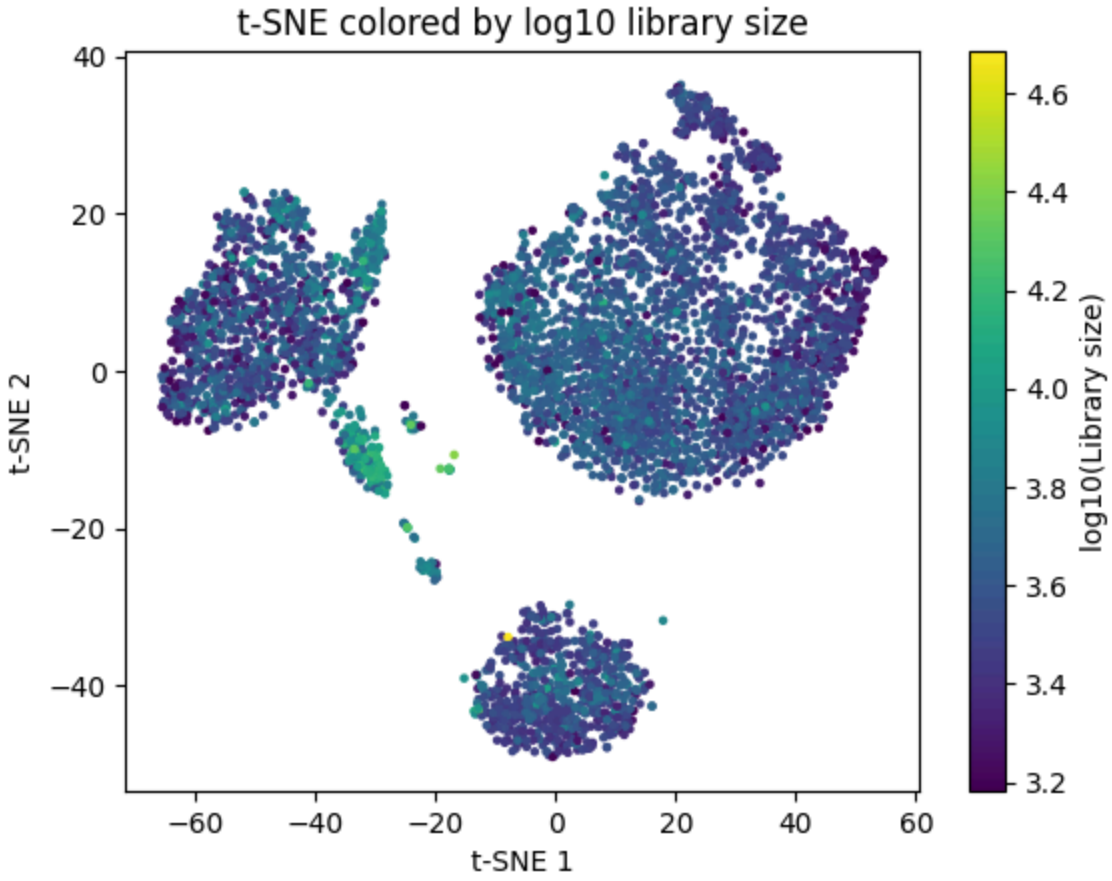
In [204…

```
# Log10 library size for each cell
log_library_size = np.log10(library_size_final)

# Plot t-SNE, colored by library size
plt.scatter(
    X_tsne[:, 0],
    X_tsne[:, 1],
    c=log_library_size,
```

```
    s=5
)

plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.title("t-SNE colored by log10 library size")
plt.colorbar(label="log10(Library size)")
plt.show()
```



I colored the t-SNE plot by log10 library size using the genes that remained after filtering. Most cells are blue or light blue, and the main groups contain a mix of library sizes. One small group around t-SNE coordinates (-30, -10) has more cells with higher library sizes. Overall, library size does not seem to explain the main group separation, but it may be related to some smaller local patterns. This plot alone cannot tell us whether those differences are technical or due to biological differences in total RNA content.

Question 2d

In [205…

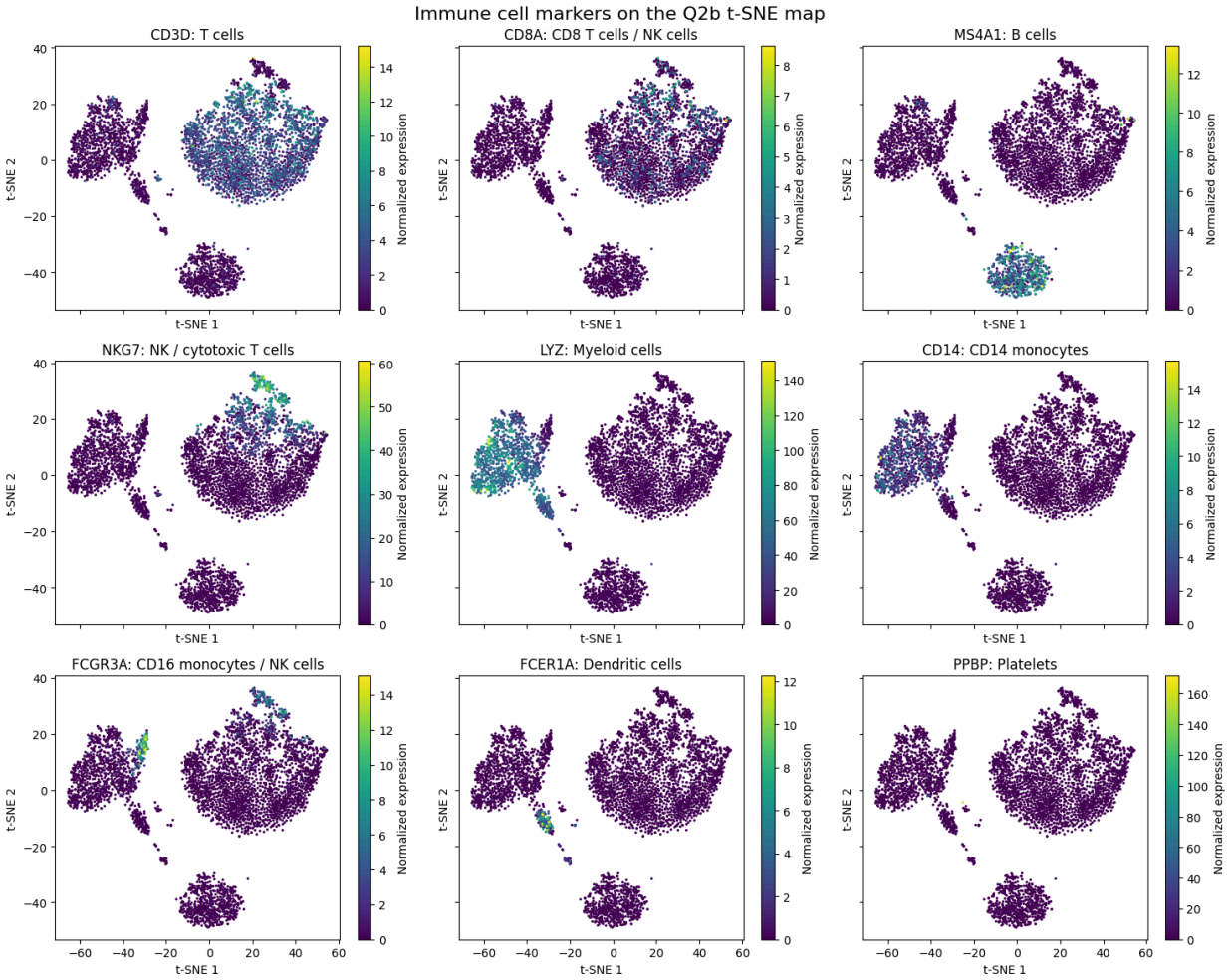
```
# Match marker names to the gene rows
marker_gene_names = genes.loc[keep_genes, 1].to_numpy()
markers = {
    "CD3D": "T cells",
    "CD8A": "CD8 T cells / NK cells",
    "MS4A1": "B cells",
    "NKG7": "NK / cytotoxic T cells",
    "LYZ": "Myeloid cells",
```

```
    "CD14": "CD14 monocytes",
    "FCGR3A": "CD16 monocytes / NK cells",
    "FCER1A": "Dendritic cells",
    "PPBP": "Platelets",
}

fig, axes = plt.subplots(3, 3, figsize=(15, 12), sharex=True, sharey=True)
for ax, (marker, cell_type) in zip(axes.flat, markers.items()):
    marker_rows = np.flatnonzero(marker_gene_names == marker)
    if len(marker_rows) == 0:
        ax.set_title(f"{marker}: not retained after filtering")
        ax.set_axis_off()
        continue

    # Use the median-library-size normalized expression from Q2a
    expression = X_normalized[marker_rows, :].sum(axis=0)
    order = np.argsort(expression)  # Draw expressing cells on top
    points = ax.scatter(
        X_tsne[order, 0], X_tsne[order, 1],
        c=expression[order], cmap="viridis", s=5, vmin=0,
        linewidths=0
    )
    ax.set_title(f"{marker}: {cell_type}")
    ax.set_xlabel("t-SNE 1")
    ax.set_ylabel("t-SNE 2")
    fig.colorbar(points, ax=ax, label="Normalized expression")

fig.suptitle("Immune cell markers on the Q2b t-SNE map", fontsize=16)
plt.tight_layout()
plt.show()
```



The large group on the right expresses CD3D, supporting T cells. CD8A expression within this group supports a CD8 T-cell subset. The small groups at the top show strong NKG7 and FCGR3A with relatively little CD3D, consistent with NK cells; NKG7 can also be expressed by cytotoxic T cells. The separate group at the bottom expresses MS4A1, identifying B cells. On the left, LYZ and CD14 identify classical monocytes, while the FCGR3A-high, relatively CD14-low region supports CD16 monocytes. The smaller group below the main monocyte group expresses FCER1A, consistent with dendritic cells. These assignments use combinations of markers because individual markers are not completely cell-type specific.

PPBP expression is restricted to very few cells, suggesting possible platelets, but it does not establish a clear platelet cluster here. FCER1A and PPBP are commonly used PBMC markers for dendritic cells and platelets, respectively. Overall, the clearest populations are T cells, B cells, NK cells, and monocytes, with a smaller dendritic-cell population.

Question 3

In [206…

```
# Select genes that are unusually variable for their average expression
n_hvg = 2000
gene_mean = X_normalized.mean(axis=1)
gene_variance = X_normalized.var(axis=1, ddof=1)

# Dispersion = variance / mean; log1p reduces the influence of extreme values
```

```
log_dispersion = pd.Series(np.log1p(gene_variance / gene_mean))

# Compare genes within 20 groups of similar average expression
mean_groups = pd.qcut(gene_mean, q=20, labels=False, duplicates="drop")
grouped = log_dispersion.groupby(mean_groups)
hvg_scores = (log_dispersion - grouped.transform("mean")) / grouped.transform('

hvg_indices = hvg_scores.nlargest(n_hvg).index.to_numpy()
X_hvg = X_normalized[hvg_indices, :]

print("HVG matrix shape:", X_hvg.shape)
```

```
HVG matrix shape: (2000, 4292)
```

In [207…

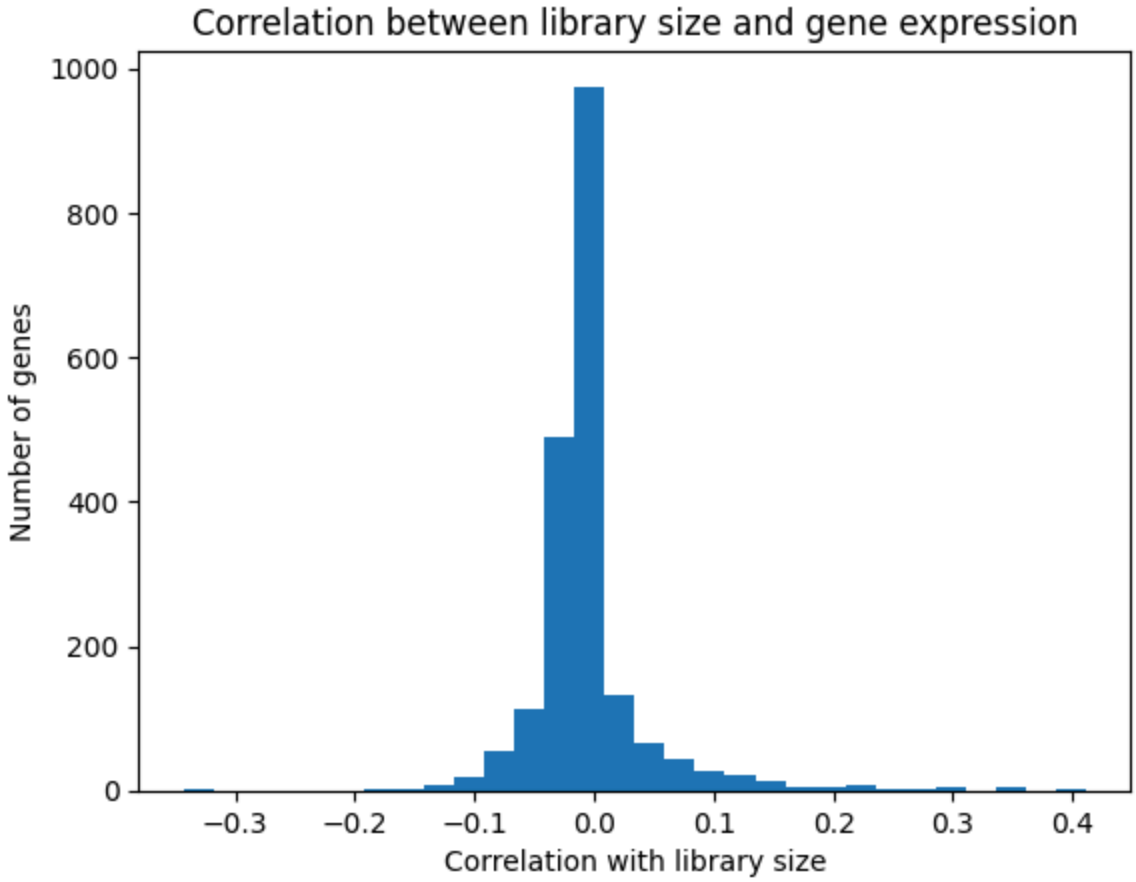
```
# Calculate correlation with library size
from scipy.stats import pearsonr

correlations = []

for i in range(X_hvg.shape[0]):
    r, p = pearsonr(library_size_final, X_hvg[i, :])
    correlations.append(r)

correlations = np.array(correlations)

plt.hist(correlations, bins=30)
plt.xlabel("Correlation with library size")
plt.ylabel("Number of genes")
plt.title("Correlation between library size and gene expression")
plt.show()
```



In [208…

```
# Fit regression and regress out library size
from sklearn.linear_model import LinearRegression

library_predictor = library_size_final.reshape(-1, 1)

X_hvg_regressed = np.zeros_like(X_hvg, dtype=float)

for i in range(X_hvg.shape[0]):
    y = X_hvg[i, :]

    model = LinearRegression()
    model.fit(library_predictor, y)

    predicted = model.predict(library_predictor)

    # Keep the residuals
    X_hvg_regressed[i, :] = y - predicted
```

In [209…

```
# Redo Q2b --PCA
X_hvg_cells = X_hvg_regressed.T

pca_regressed = PCA(n_components=2)
X_pca_regressed = pca_regressed.fit_transform(X_hvg_cells)

plt.scatter(X_pca_regressed[:, 0], X_pca_regressed[:, 1], s=5)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA after regressing out library size")
plt.show()
```



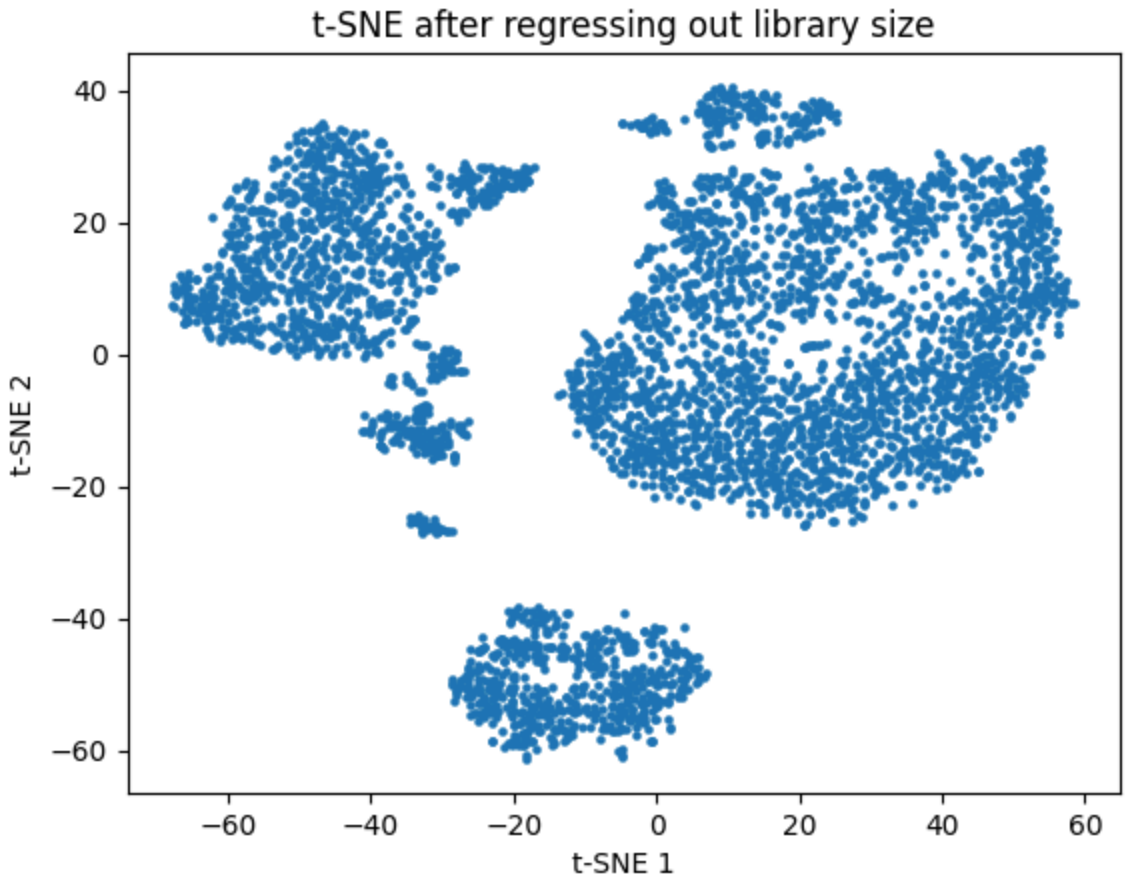
In [210…

```
# Redo Q2b -- t-SNE
tsne_regressed = TSNE(
    n_components=2,
    random_state=42,
    perplexity=30
)

X_tsne_regressed = tsne_regressed.fit_transform(X_hvg_cells)

plt.scatter(
    X_tsne_regressed[:, 0],
    X_tsne_regressed[:, 1],
    s=5
)

plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.title("t-SNE after regressing out library size")
plt.show()
```

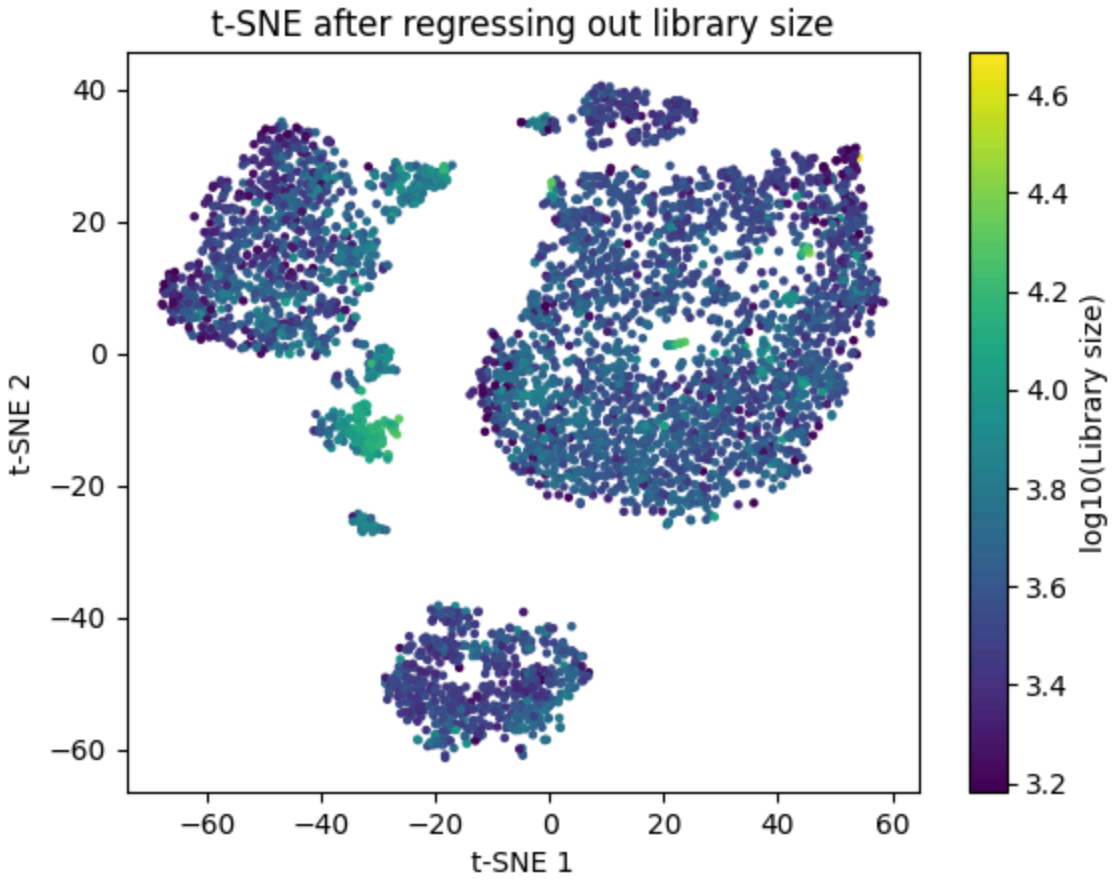


In [211…

```
# Color it
plt.scatter(
    X_tsne_regressed[:, 0],
    X_tsne_regressed[:, 1],
    c=np.log10(library_size_final),
    s=5
)

plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
```

```
plt.title("t-SNE after regressing out library size")
plt.colorbar(label="log10(Library size)")
plt.show()
```



Among the 2,000 selected genes, most correlations with library size were close to zero, but some genes had an absolute correlation above 0.2. I fitted a regression for each gene and subtracted the predicted expression to remove the remaining linear relationship. I did not use a regularizer because each model had only one predictor, library size, and 4,292 cells, so there is little risk of overfitting the regression coefficients.

After correction, PCA still showed overlapping cells, while t-SNE showed clearer groups. Some smaller regions still had higher library sizes, suggesting that regression did not remove every library-size-related pattern. Differences from Q2 may also reflect the smaller set of genes used here.

Question 4: PRML 2.13

For two multivariate Gaussian distributions,

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

and

$$q(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \mathbf{m}, \mathbf{L}),$$

the KL divergence is

$$KL(p\|q) = \int p(\mathbf{x}) \ln \frac{p(\mathbf{x})}{q(\mathbf{x})}\, d\mathbf{x}.$$

For a D-dimensional Gaussian, write the log densities,

$$\ln p(\mathbf{x}) = -\frac{D}{2}\ln(2\pi) - \frac{1}{2}\ln|\boldsymbol{\Sigma}| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}),$$

and

$$\ln q(\mathbf{x}) = -\frac{D}{2}\ln(2\pi) - \frac{1}{2}\ln|\mathbf{L}| - \frac{1}{2}(\mathbf{x}-\mathbf{m})^T\mathbf{L}^{-1}(\mathbf{x}-\mathbf{m}).$$

Therefore,

$$\ln \frac{p(\mathbf{x})}{q(\mathbf{x})} = \frac{1}{2}\ln\frac{|\mathbf{L}|}{|\boldsymbol{\Sigma}|} - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) + \frac{1}{2}(\mathbf{x}-\mathbf{m})^T\mathbf{L}^{-1}(\mathbf{x}-\mathbf{m}).$$

Take the expectation under p, using

$$E_p[(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})] = D,$$

and

$$E_p[(\mathbf{x}-\mathbf{m})^T\mathbf{L}^{-1}(\mathbf{x}-\mathbf{m})] = \mathrm{tr}(\mathbf{L}^{-1}\boldsymbol{\Sigma}) + (\mathbf{m}-\boldsymbol{\mu})^T\mathbf{L}^{-1}(\mathbf{m}-\boldsymbol{\mu}),$$

we obtain

$$\boxed{KL(p\|q) = \frac{1}{2}\left[\ln\frac{|\mathbf{L}|}{|\boldsymbol{\Sigma}|} - D + \mathrm{tr}(\mathbf{L}^{-1}\boldsymbol{\Sigma}) + (\mathbf{m}-\boldsymbol{\mu})^T\mathbf{L}^{-1}(\mathbf{m}-\boldsymbol{\mu})\right]}$$

Question 5 — PRML 3.12

Let N be the number of observations and M the number of weights. The likelihood is

$$p(\mathbf{t} \mid \mathbf{w}, \beta) = \mathcal{N}(\mathbf{t} \mid \boldsymbol{\Phi}\mathbf{w}, \beta^{-1}\mathbf{I}_N).$$

The prior is

$$p(\mathbf{w}, \beta) = \mathcal{N}(\mathbf{w} \mid \mathbf{m}_0, \beta^{-1}\mathbf{S}_0)\,\mathrm{Gam}(\beta \mid a_0, b_0),$$

where $\mathbf{S}_0$ is a positive-definite covariance factor. I use the shape-rate convention, so the gamma density is proportional to $\beta^{a_0-1}\exp(-b_0\beta)$.

By Bayes' rule, retaining all factors that depend on $\mathbf{w}$ or β,

$$p(\mathbf{w}, \beta \mid \mathbf{t}) \propto \beta^{a_0-1+(N+M)/2}\exp\left\{-\beta\left[b_0 + \frac{1}{2}Q(\mathbf{w})\right]\right\},$$

where

$$\begin{aligned} Q(\mathbf{w}) &= (\mathbf{t}-\boldsymbol{\Phi}\mathbf{w})^T(\mathbf{t}-\boldsymbol{\Phi}\mathbf{w}) + (\mathbf{w}-\mathbf{m}_0)^T\mathbf{S}_0^{-1}(\mathbf{w}-\mathbf{m}_0) \\ &= \mathbf{w}^T(\boldsymbol{\Phi}^T\boldsymbol{\Phi}+\mathbf{S}_0^{-1})\mathbf{w} - 2\mathbf{w}^T(\boldsymbol{\Phi}^T\mathbf{t}+\mathbf{S}_0^{-1}\mathbf{m}_0) + \mathbf{t}^T\mathbf{t} + \mathbf{m}_0^T\mathbf{S}_0^{-1}\mathbf{m}_0. \end{aligned}$$

Define

$$\boxed{\mathbf{S}_N = (\mathbf{S}_0^{-1}+\boldsymbol{\Phi}^T\boldsymbol{\Phi})^{-1}}, \qquad \boxed{\mathbf{m}_N = \mathbf{S}_N(\mathbf{S}_0^{-1}\mathbf{m}_0+\boldsymbol{\Phi}^T\mathbf{t})}.$$

Completing the square gives

$$Q(\mathbf{w}) = (\mathbf{w}-\mathbf{m}_N)^T\mathbf{S}_N^{-1}(\mathbf{w}-\mathbf{m}_N) + \mathbf{t}^T\mathbf{t} + \mathbf{m}_0^T\mathbf{S}_0^{-1}\mathbf{m}_0 - \mathbf{m}_N^T\mathbf{S}_N^{-1}\mathbf{m}_N.$$

Therefore, with

$$\boxed{a_N = a_0 + \frac{N}{2}},$$

$$\boxed{b_N = b_0 + \frac{1}{2}\left(\mathbf{t}^T\mathbf{t} + \mathbf{m}_0^T\mathbf{S}_0^{-1}\mathbf{m}_0 - \mathbf{m}_N^T\mathbf{S}_N^{-1}\mathbf{m}_N\right)},$$

the posterior kernel factors as

$$p(\mathbf{w},\beta \mid \mathbf{t}) \propto \underbrace{\beta^{M/2}\exp\left[-\frac{\beta}{2}(\mathbf{w}-\mathbf{m}_N)^T\mathbf{S}_N^{-1}(\mathbf{w}-\mathbf{m}_N)\right]}_{\text{conditional Gaussian kernel}} \underbrace{\beta^{a_N-1}\exp(-b_N\beta)}_{\text{gamma kernel}}.$$

The Gaussian factor accounts for the $\beta^{M/2}$ term, leaving $a_N = a_0 + N/2$ for the gamma shape. Normalizing these factors yields

$$\boxed{p(\mathbf{w},\beta \mid \mathbf{t}) = \mathcal{N}(\mathbf{w} \mid \mathbf{m}_N, \beta^{-1}\mathbf{S}_N)\,\mathrm{Gam}(\beta \mid a_N, b_N)}.$$

The posterior has the same normal-gamma form as the prior, establishing conjugacy.